

สัปดาห์ที่ 4

การสร้าง Star Schema และ Metric Layer ด้วย dbt

กรณีศึกษา Coffee Club | ระยะเวลา 2 ชั่วโมง

แนวคิดของ Lab

เปลี่ยนการสร้างตารางด้วย SQL แบบทีละขั้น ให้เป็นกระบวนการแปลงข้อมูลที่จัดการด้วย dbt และต่อยอด Star Schema ไปสู่ Metric Layer ซึ่งรวมตรรกะการคำนวณ KPI ไว้เป็นศูนย์กลางสำหรับ Metabase

วัตถุประสงค์

- วิเคราะห์ข้อมูลจากไฟล์ CSV เพื่อออกแบบ Star Schema ได้
- แยก Fact และ Dimension ออกจากข้อมูลต้นทาง และสร้างเป็น dbt models ได้
- กำหนด source, staging, marts และการทดสอบคุณภาพข้อมูลใน dbt ได้
- สร้าง Metric Layer จาก KPI ทางธุรกิจ

แผนการทำงาน

ช่วงเวลา	กิจกรรม
0-20 นาที	วิเคราะห์ข้อมูลและออกแบบ Star Schema
20-40 นาที	เตรียมฐานข้อมูลต้นทางและตั้งค่า dbt source
40-75 นาที	สร้าง staging, dimension และ fact models
75-105 นาที	สร้าง Metric Layer จาก KPI ที่กำหนด
105-120 นาที	ทดสอบ dbt models และตรวจสอบผลลัพธ์ของ KPI

Dataset

ชื่อไฟล์: coffee_sales.csv

กรณีศึกษา: Coffee Club – ระบบข้อมูลการขายสำหรับร้านกาแฟระบบสมาชิก

บริบทธุรกิจ (Business Context)

Coffee Club เป็นร้านกาแฟที่มีหลายสาขาทั่วประเทศไทย โดยมีระบบสมาชิกสะสมแต้ม ระบบโปรโมชั่นตามฤดูกาล และระบบพนักงานประจำสาขา การขายเกิดขึ้นได้ทั้งช่วงปกติและระหว่างจัดโปรโมชั่น โดยแต่ละการขายจะบันทึกข้อมูลลูกค้า สินค้า พนักงานที่ให้บริการ โปรโมชั่นที่ใช้ (ถ้ามี) และแต้มที่แลก ข้อมูลถูกเก็บในรูปแบบแฟลตไฟล์ (coffee_sales.csv) ซึ่งประกอบด้วยข้อมูลทุกอย่างในแถวเดียว บริษัทต้องการนำข้อมูลเข้าสู่ Data Warehouse เพื่อวิเคราะห์ยอดขาย ประสิทธิภาพของโปรโมชั่น และพฤติกรรมลูกค้า

Data Dictionary

column_name	Description
sale_id	รหัสประจำธุรกรรมการขายแต่ละรายการ (Primary Key)
invoice_number	หมายเลขใบเสร็จที่ใช้ในธุรกรรม
sale_date	วันที่ที่มีการขายสินค้าเกิดขึ้น
customer_code	รหัสลูกค้าที่ได้จากระบบต้นทาง
customer_name	ชื่อลูกค้า
gender	เพศของลูกค้า (M = ชาย, F = หญิง)
birth_year	ปีเกิดของลูกค้า
product_code	รหัสประจำสินค้าที่ขาย
product_name	ชื่อสินค้าที่ขาย
category	ประเภทของสินค้า เช่น กาแฟ ชา หรือเบเกอรี่
size	ขนาดของสินค้า (S, M, L หรือ '-')
unit_price	ราคาต่อหน่วยของสินค้า
quantity	จำนวนสินค้าที่ขายในรายการนี้
revenue	จำนวนเงินที่ลูกค้าชำระจริงหลังหักส่วนลดแล้ว
store_code	รหัสร้านค้าที่ขายสินค้า
store_name	ชื่อของร้านค้า
province	จังหวัดที่ตั้งของร้านค้า
staff_code	รหัสพนักงานที่ดูแลการขาย

column_name	Description
staff_name	ชื่อของพนักงาน
position	ตำแหน่งของพนักงาน เช่น Barista หรือ Cashier
promo_code	รหัสโปรโมชั่นที่ใช้ (ถ้ามี)
promo_desc	คำอธิบายของโปรโมชั่นที่ใช้
points_redeemed	จำนวนแต้มสะสมที่ลูกค้าใช้แลกในรายการนี้

Business Processes ที่เกี่ยวข้อง

Business Process	รายละเอียด
Make a Sale	ลูกค้าซื้อสินค้าในร้าน สะสมแต้ม หรือแลกแต้ม
Apply Promotion	ใช้โปรโมชั่นลดราคา ณ จุดขาย
Serve by Staff	พนักงานบริการลูกค้าแต่ละคนในแต่ละบิล

KPI ที่ต้องการ

metric_key	Business Process	KPI	Grain ของผลลัพธ์
M001	Make a Sale	ยอดขายรวม	เดือน
M002	Make a Sale	ยอดขายต่อบิล	เดือน + invoice_number
M003	Make a Sale	ยอดขายต่อสินค้า	เดือน + product_code
M004	Apply Promotion	มูลค่าส่วนลดจากการใช้โปรโมชั่น	เดือน + promo_code
M005	Apply Promotion	จำนวนบิลที่ใช้โปรโมชั่น	เดือน + promo_code
M006	Apply Promotion	แต้มที่แลกใช้แยกตามลูกค้า	เดือน + customer_code
M007	Serve by Staff	ยอดขายต่อพนักงาน	เดือน + staff_code
M008	Serve by Staff	ประเภทสินค้าที่ขายมากที่สุดของพนักงานแต่ละคน	เดือน + staff_code

สถาปัตยกรรมข้อมูลที่ใช้

ลำดับ	Layer	หน้าที่
1	CSV / PostgreSQL raw table	ข้อมูลต้นทาง coffee_staging
2	Staging Layer	ปรับชื่อคอลัมน์ ชนิดข้อมูล ค่า null และข้อมูลโปรโมชั่น
3	Core Data Warehouse	Dimension tables และ fct_sales ตาม Star Schema
4	Metric Layer	คำนวณ M001-M008 ด้วยนิยามเดียวกัน

โครงสร้าง dbt project

โครงสร้างไฟล์ที่แนะนำ

models/ ├── staging/

```
|   ├── _sources.yml
|   ├── stg_coffee_sales.sql
|   ├── dimensions/
|   |   ├── dim_date.sql
|   |   ├── dim_customer.sql
|   |   ├── dim_product.sql
|   |   ├── dim_store.sql
|   |   ├── dim_staff.sql
|   |   └── dim_promotion.sql
|   ├── marts/
|   |   └── fct_sales.sql
|   ├── metrics/
|   |   ├── metric_m001_total_revenue.sql
|   |   ├── metric_m002_revenue_by_invoice.sql
|   |   └── metric_m003...m008.sql
|   └── schema.yml
tests/
└── assert_fct_sales_row_count.sql
```

คำสั่ง

ขั้นที่ 1: ออกแบบ Star Schema

1. วิเคราะห์ข้อมูลของ Coffee Club เพื่อกำหนด grain ของ Fact Table และ Dimension Tables ที่จำเป็นต่อ KPI
2. เขียน ER-Diagram แสดง Star Schema พร้อม Primary Key, Foreign Key และความสัมพันธ์ให้ครบทุกตาราง
3. ระบุว่าแต่ละ KPI จะอ่านข้อมูลจาก fact/dimension ใด และมี grain ของผลลัพธ์ระดับใด

ขั้นที่ 2: เตรียมฐานข้อมูลต้นทางใน PostgreSQL

dbt ทำหน้าที่ Transform ข้อมูล ไม่ได้ทำหน้าที่นำเข้าไฟล์ CSV เข้าฐานข้อมูล ดังนั้นให้โหลดไฟล์เข้าสู่ raw table ก่อน

2.1 สร้างฐานข้อมูลผ่าน pgAdmin SQL Editor

```
CREATE DATABASE coffee_dw;
```

2.2 สร้าง raw table

```
CREATE TABLE public.coffee_staging (
  sale_id INT,
  invoice_number VARCHAR,
  sale_date DATE,
  customer_code VARCHAR,
  customer_name VARCHAR,
  gender CHAR(1),
  birth_year INT,
  product_code VARCHAR,
  product_name VARCHAR,
  category VARCHAR,
  size VARCHAR,
  unit_price NUMERIC,
  quantity INT,
  revenue NUMERIC,
  store_code VARCHAR,
  store_name VARCHAR,
  province VARCHAR,
  staff_code VARCHAR,
  staff_name VARCHAR,
  position VARCHAR,
  promo_code VARCHAR,
  promo_desc VARCHAR,
  points_redeemed INT
);
```

2.3 ใช้ Import/Export Data ของ pgAdmin เพื่อนำ coffee_sales.csv เข้าตาราง public.coffee_staging แล้วตรวจสอบจำนวนแถว

```
SELECT COUNT(*) AS row_count FROM public.coffee_staging;
```

ขั้นที่ 3: ตรวจสอบการเชื่อมต่อ dbt

สร้าง profile ชื่อ coffee_dw กำหนดให้ database = coffee_dw จากนั้นรันคำสั่งต่อไปนี้

```
docker exec -it dw_dbt bash
export DBT_PROFILE=coffee_dw
```

```
dbt debug
```

ขั้นที่ 4: ตั้งค่า dbt project และ Source

4.1 dbt_project.yml

```
name: 'coffee_dw'
version: '1.0.0'
config-version: 2
profile: 'coffee_dw'

model-paths: ['models']

models:
  coffee_dw:
    staging:
      +materialized: view
    dimensions:
      +materialized: table
    marts:
      +materialized: table
    metrics:
      +materialized: view
```

ทดสอบด้วยการรันคำสั่งต่อไปนี้ใน container dw_dbt

```
cd coffee_dw
dbt parse
```

4.2 models/coffee_dw/_sources.yml

```
version: 2

sources:
  - name: raw
    schema: public
    tables:
      - name: coffee_staging
        columns:
          - name: sale_id
            data_tests:
              - not_null
              - unique
          - name: sale_date
            data_tests:
              - not_null
```

ขั้นที่ 5: สร้าง Staging Model

models/coffee_dw/staging/stg_coffee_sales.sql

```
select
  sale_id::integer                as sale_id,
  nullif(trim(invoice_number), '') as invoice_number,
  sale_date::date                 as sale_date,
  nullif(trim(customer_code), '') as customer_code,
  nullif(trim(customer_name), '') as customer_name,
  upper(nullif(trim(gender), '')) as gender,
  birth_year::integer             as birth_year,
  nullif(trim(product_code), '')  as product_code,
  nullif(trim(product_name), '')  as product_name,
  nullif(trim(category), '')      as category,
  nullif(trim(size), '')          as size,
  unit_price::numeric             as unit_price,
  quantity::integer               as quantity,
  revenue::numeric                as revenue,
  nullif(trim(store_code), '')    as store_code,
```

```

nullif(trim(store_name), "") as store_name,
nullif(trim(province), "") as province,
nullif(trim(staff_code), "") as staff_code,
nullif(trim(staff_name), "") as staff_name,
nullif(trim(position), "") as position,
coalesce(nullif(trim(promo_code), ''), 'NO_PROMO') as promo_code,
coalesce(nullif(trim(promo_desc), ''), 'ไม่มีโปรโมชั่น') as promo_desc,
coalesce(points_redeemed, 0)::integer as points_redeemed
from {{ source('raw', 'coffee_staging') }}

```

ขั้นที่ 6: สร้าง Dimension Models

สร้าง Dimension Tables ตาม Star Schema ที่ออกแบบไว้ โดยใช้ business key จากระบบต้นทาง และสร้าง surrogate key แบบคณที่ด้วย md5()

models/coffee_dw/dimensions/dim_date.sql

```

select distinct
  to_char(sale_date, 'YYYYMMDD')::integer as date_key,
  sale_date as full_date,
  extract(day from sale_date)::integer as day,
  extract(month from sale_date)::integer as month,
  trim(to_char(sale_date, 'Month')) as month_name,
  extract(quarter from sale_date)::integer as quarter,
  extract(year from sale_date)::integer as year
from {{ ref('stg_coffee_sales') }}

```

ตัวอย่าง: models/coffee_dw/dimensions/dim_customer.sql

```

select distinct
  md5(coalesce(customer_code, '__NULL__')) as customer_key,
  customer_code,
  customer_name,
  gender,
  birth_year
from {{ ref('stg_coffee_sales') }}

```

Model	Business key	Attributes ที่ควรมี
dim_date	sale_date	date_key, full_date, day, month, month_name, quarter, year
dim_customer	customer_code	customer_name, gender, birth_year
dim_product	product_code	product_name, category, size
dim_store	store_code	store_name, province
dim_staff	staff_code	staff_name, position
dim_promotion	promo_code	promo_desc; ต้องมี NO_PROMO

ขั้นที่ 7: สร้าง Fact Model

กำหนด grain ของ fct_sales ให้ 1 แถวแทน 1 รายการขาย (sale_id) จากนั้น join staging กับทุก dimension เพื่อแทน business key ด้วย surrogate key

models/coffee_dw/marts/fct_sales.sql

```
select
  s.sale_id,
  s.invoice_number,
  d.date_key,
  c.customer_key,
  p.product_key,
  st.store_key,
  sf.staff_key,
  pr.promotion_key,
  s.unit_price,
  s.quantity,
  s.revenue,
  (s.unit_price * s.quantity) - s.revenue as discount_amount,
  s.points_redeemed
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_date') }} d
  on s.sale_date = d.full_date
join {{ ref('dim_customer') }} c
  on s.customer_code = c.customer_code
join {{ ref('dim_product') }} p
  on s.product_code = p.product_code
join {{ ref('dim_store') }} st
  on s.store_code = st.store_code
join {{ ref('dim_staff') }} sf
  on s.staff_code = sf.staff_code
join {{ ref('dim_promotion') }} pr
  on s.promo_code = pr.promo_code
```

ข้อควรระวัง

ทุก business key ใน Dimension ต้องไม่ซ้ำ มีฉะนั้นการ join จะทำให้จำนวนแถวใน Fact มากกว่าข้อมูล Staging ควรตรวจด้วย unique test และ singular test ในขั้นที่ 9

ขั้นที่ 8: สร้าง Metric Layer

สร้าง dbt model แยกตาม metric_key เพื่อให้สูตรคำนวณและ grain ของแต่ละ KPI ชัดเจน
ชื่อคอลัมน์ผลลัพธ์ใช้รูปแบบร่วมกันเท่าที่ทำได้ ได้แก่ metric_key, metric_month, dimension_key, dimension_name และ metric_value

ตัวอย่าง: metric_m001_total_revenue.sql

```
select
  'M001'::varchar as metric_key,
  date_trunc('month', d.full_date)::date as metric_month,
  'ALL'::varchar as dimension_key,
  'Coffee Club'::varchar as dimension_name,
  sum(f.revenue)::numeric as metric_value
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d using (date_key)
group by 1, 2, 3, 4
```

แม่แบบสำหรับ M002-M007 (ข้อความใน <...> เป็นส่วนที่ผู้เรียนต้องแทนค่า)

```
select
  '<metric_key>'::varchar as metric_key,
  date_trunc('month', d.full_date)::date as metric_month,
  <dimension_key> as dimension_key,
  <dimension_name> as dimension_name,
  <aggregate_expression> as metric_value
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d using (date_key)
-- ผู้เรียนเติม dimension join, filter และ group by ให้ตรงกับ grain ของ KPI
```

Key	การคำนวณ	Dimension/เงื่อนไข	ชื่อ model
M001	SUM(revenue)	บริษัท	metric_m001_total_revenue.sql
M002	SUM(revenue)	invoice_number	metric_m002_revenue_by_invoice.sql
M003	SUM(revenue)	product_code	metric_m003_revenue_by_product.sql
M004	SUM(discount_amount)	promo_code; ไม่รวม NO_PROMO	metric_m004_discount_by_promotion.sql
M005	COUNT(DISTINCT invoice_number)	promo_code; ไม่รวม NO_PROMO	metric_m005_promo_bill_count.sql
M006	SUM(points_redeemed)	customer_code	metric_m006_points_by_customer.sql
M007	SUM(revenue)	staff_code	metric_m007_revenue_by_staff.sql
M008	SUM(quantity) แล้วจัดอันดับ	staff_code + category; เลือกอันดับ 1	metric_m008_top_category_by_staff.sql

แนวทางสำหรับ M008

models/coffee_dw/metrics/metric_m008_top_category_by_staff.sql

```

with category_sales as (
  select
    date_trunc('month', d.full_date)::date as metric_month,
    sf.staff_code,
    sf.staff_name,
    p.category,
    sum(f.quantity)::numeric as quantity_sold
  from {{ ref('fct_sales') }} f
  join {{ ref('dim_date') }} d using (date_key)
  join {{ ref('dim_staff') }} sf using (staff_key)
  join {{ ref('dim_product') }} p using (product_key)
  group by 1, 2, 3, 4
), ranked as (
  select
    *,
    row_number() over (
      partition by metric_month, staff_code
      order by quantity_sold desc, category
    ) as category_rank
  from category_sales
)
select
  'M008'::varchar as metric_key,
  metric_month,
  staff_code::varchar as dimension_key,
  staff_name::varchar as dimension_name,
  category::varchar as result_label,
  quantity_sold as metric_value
from ranked
where category_rank = 1

```

สำหรับ M008 ค่า dimension_key และ dimension_nameระบุพนักงาน, result_labelระบุประเภทสินค้าที่ได้อันดับ 1 และ metric_value คือจำนวนสินค้าที่ขายได้ในประเภทนั้น

ขั้นที่ 9: กำหนด Data Tests

models/coffee_dw/schema.yml

```
version: 2

models:
  - name: stg_coffee_sales
    description: "ข้อมูลการขายที่ป้อนชนิดข้อมูลและจัดการค่าว่างแล้ว"
    columns:
      - name: sale_id
        description: "รหัสรายการขาย"
        data_tests: [not_null, unique]
      - name: invoice_number
        description: "หมายเลขใบเสร็จ"
        data_tests: [not_null]
      - name: sale_date
        description: "วันที่ขาย"
        data_tests: [not_null]
      - name: customer_code
        description: "รหัสลูกค้า"
        data_tests: [not_null]
      - name: product_code
        description: "รหัสสินค้า"
        data_tests: [not_null]
      - name: store_code
        description: "รหัสสาขา"
        data_tests: [not_null]
      - name: staff_code
        description: "รหัสพนักงาน"
        data_tests: [not_null]
      - name: promo_code
        description: "รหัสโปรโมชั่น หรือ NO_PROMO"
        data_tests: [not_null]

  - name: dim_date
    description: "มิติวันที่"
    columns:
      - name: date_key
        description: "Surrogate key ของวันที่ รูปแบบ YYYYMMDD"
        data_tests: [not_null, unique]
      - name: full_date
        description: "วันที่ปฏิทิน"
        data_tests: [not_null, unique]

  - name: dim_customer
    description: "มิติลูกค้า"
    columns:
      - name: customer_key
        description: "Surrogate key ของลูกค้า"
        data_tests: [not_null, unique]
      - name: customer_code
        description: "Business key ของลูกค้า"
        data_tests: [not_null, unique]

  - name: dim_product
    description: "มิติสินค้า"
    columns:
      - name: product_key
        description: "Surrogate key ของสินค้า"
        data_tests: [not_null, unique]
      - name: product_code
        description: "Business key ของสินค้า"
        data_tests: [not_null, unique]

  - name: dim_store
    description: "มิติสาขา"
    columns:
      - name: store_key
        description: "Surrogate key ของสาขา"
        data_tests: [not_null, unique]
      - name: store_code
        description: "Business key ของสาขา"
        data_tests: [not_null, unique]

  - name: dim_staff
    description: "มิติพนักงาน"
    columns:
      - name: staff_key
        description: "Surrogate key ของพนักงาน"
        data_tests: [not_null, unique]
      - name: staff_code
        description: "Business key ของพนักงาน"
        data_tests: [not_null, unique]

  - name: dim_promotion
```

```

description: "มิติโปรโมชั่น รวมสมาชิก NO_PROMO"
columns:
  - name: promotion_key
    description: "Surrogate key ของโปรโมชั่น"
    data_tests: [not_null, unique]
  - name: promo_code
    description: "Business key ของโปรโมชั่น"
    data_tests: [not_null, unique]

- name: fct_sales
  description: "Fact การขาย โดย 1 แถวแทน 1 รายการขาย"
  columns:
    - name: sale_id
      description: "รหัสรายการขายและ degenerate key ของ Fact"
      data_tests: [not_null, unique]
    - name: invoice_number
      description: "หมายเลขใบเสร็จ"
      data_tests: [not_null]
    - name: date_key
      data_tests:
        - not_null
        - relationships:
            to: ref('dim_date')
            field: date_key
    - name: customer_key
      data_tests:
        - not_null
        - relationships:
            to: ref('dim_customer')
            field: customer_key
    - name: product_key
      data_tests:
        - not_null
        - relationships:
            to: ref('dim_product')
            field: product_key
    - name: store_key
      data_tests:
        - not_null
        - relationships:
            to: ref('dim_store')
            field: store_key
    - name: staff_key
      data_tests:
        - not_null
        - relationships:
            to: ref('dim_staff')
            field: staff_key
    - name: promotion_key
      data_tests:
        - not_null
        - relationships:
            to: ref('dim_promotion')
            field: promotion_key
    - name: quantity
      data_tests: [not_null]
    - name: revenue
      data_tests: [not_null]
    - name: discount_amount
      data_tests: [not_null]
    - name: points_redeemed
      data_tests: [not_null]

- name: metric_m001_total_revenue
  description: "M001 ยอดขายรวมรายเดือน"
  columns:
    - name: metric_key
      data_tests:
        - not_null
        - accepted_values:
            values: ['M001']
    - name: metric_month
      data_tests: [not_null]
    - name: dimension_key
      data_tests: [not_null]
    - name: dimension_name
      data_tests: [not_null]
    - name: metric_value
      data_tests: [not_null]

- name: metric_m002_revenue_by_invoice
  description: "M002 ยอดขายรายเดือนแยกตามใบเสร็จ"
  columns:
    - name: metric_key
      data_tests:
        - not_null
        - accepted_values:
            values: ['M002']
    - name: metric_month
      data_tests: [not_null]
    - name: dimension_key

```

```

    data_tests: [not_null]
  - name: dimension_name
    data_tests: [not_null]
  - name: metric_value
    data_tests: [not_null]

- name: metric_m003_revenue_by_product
description: "M003 ยอดขายรายเดือนแยกตามสินค้า"
columns:
  - name: metric_key
    data_tests:
      - not_null
      - accepted_values:
          values: ['M003']
  - name: metric_month
    data_tests: [not_null]
  - name: dimension_key
    data_tests: [not_null]
  - name: dimension_name
    data_tests: [not_null]
  - name: metric_value
    data_tests: [not_null]

- name: metric_m004_discount_by_promotion
description: "M004 มูลค่าส่วนลดรายเดือนแยกตามโปรโมชั่น"
columns:
  - name: metric_key
    data_tests:
      - not_null
      - accepted_values:
          values: ['M004']
  - name: metric_month
    data_tests: [not_null]
  - name: dimension_key
    data_tests: [not_null]
  - name: dimension_name
    data_tests: [not_null]
  - name: metric_value
    data_tests: [not_null]

- name: metric_m005_promo_bill_count
description: "M005 จำนวนใบเสร็จที่ใช้โปรโมชั่นรายเดือน"
columns:
  - name: metric_key
    data_tests:
      - not_null
      - accepted_values:
          values: ['M005']
  - name: metric_month
    data_tests: [not_null]
  - name: dimension_key
    data_tests: [not_null]
  - name: dimension_name
    data_tests: [not_null]
  - name: metric_value
    data_tests: [not_null]

- name: metric_m006_points_by_customer
description: "M006 แดงที่แลกใช้รายเดือนแยกตามลูกค้า"
columns:
  - name: metric_key
    data_tests:
      - not_null
      - accepted_values:
          values: ['M006']
  - name: metric_month
    data_tests: [not_null]
  - name: dimension_key
    data_tests: [not_null]
  - name: dimension_name
    data_tests: [not_null]
  - name: metric_value
    data_tests: [not_null]

- name: metric_m007_revenue_by_staff
description: "M007 ยอดขายรายเดือนแยกตามพนักงาน"
columns:
  - name: metric_key
    data_tests:
      - not_null
      - accepted_values:
          values: ['M007']
  - name: metric_month
    data_tests: [not_null]
  - name: dimension_key
    data_tests: [not_null]
  - name: dimension_name
    data_tests: [not_null]
  - name: metric_value
    data_tests: [not_null]

```

```

- name: metric_m008_top_category_by_staff
description: "M008 ประเภทสินค้าที่ขายได้มากที่สุดของพนักงานแต่ละคนในแต่ละเดือน"
columns:
  - name: metric_key
    data_tests:
      - not_null
      - accepted_values:
          values: ['M008']
  - name: metric_month
    data_tests: [not_null]
  - name: dimension_key
    description: "รหัสพนักงาน"
    data_tests: [not_null]
  - name: dimension_name
    description: "ชื่อพนักงาน"
    data_tests: [not_null]
  - name: result_label
    description: "ประเภทสินค้าที่ขายได้มากที่สุด"
    data_tests: [not_null]
  - name: metric_value
    description: "จำนวนสินค้าที่ขายได้ในประเภทอันดับหนึ่ง"
    data_tests: [not_null]

```

tests/assert_fct_sales_row_count.sql

```

with staging as (
  select count(*) as row_count
  from {{ ref('stg_coffee_sales') }}
),
fact as (
  select count(*) as row_count
  from {{ ref('fct_sales') }}
)
select staging.row_count as staging_rows,
       fact.row_count as fact_rows
from staging cross join fact
where staging.row_count <> fact.row_count

```

ขั้นที่ 10: รับ dbt และตรวจผล

คำสั่งหลัก

```

dbt build --select +fct_sales+
dbt test
dbt docs generate

```

การสร้างและเปิดดู dbt Documentation

dbt docs generate รวบรวมคำอธิบายและ tests จากไฟล์ YAML, ความสัมพันธ์จาก source() และ ref() รวมถึง metadata ของตารางและคอลัมน์จากฐานข้อมูล แล้วสร้างไฟล์เอกสารไว้ในโฟลเดอร์ target เช่น manifest.json, catalog.json และ index.html คำสั่งนี้สร้างเอกสารเท่านั้น ยังไม่เปิดเว็บไซต์

```

dbt docs generate
dbt docs serve --host 0.0.0.0 --port 8080

```

dbt docs serve เปิดเว็บสำหรับดูรายละเอียด model, column, test และ lineage graph หากรันใน Docker ต้องกำหนด ports: ["28088:8080"] ให้ service dbt แล้วเปิด http://localhost:28088 จากเบราว์เซอร์ คำสั่ง serve จะทำงานค้างไว้จนกด Ctrl+C

ให้ตรวจ lineage ว่าข้อมูลเชื่อมจาก coffee_staging ผ่าน staging และ dimensions/fct_sales ไปยัง metric models พร้อมเปิดดูคำอธิบายและผล tests ที่กำหนดใน schema.yml

จุดตรวจ	ผลที่คาดหวัง
Source tests	sale_id ไม่เป็น null และไม่ซ้ำ
Dimension tests	business key และ surrogate key ไม่ซ้ำ
Fact row count	จำนวนแถว fct_sales เท่ากับ stg_coffee_sales
Relationship tests	Foreign key ทุกค่าอ้างอิง Dimension ได้
Metric values	ค่า KPI ตรงกับ SQL ตรวจสอบอย่างน้อย 2 รายการ

สิ่งที่ต้องส่ง

ส่งคำตอบใน Google Classroom หัวข้อ **Lab 4: การวิเคราะห์ ออกแบบ และสร้าง Star Schema**